California
Govind Sharma

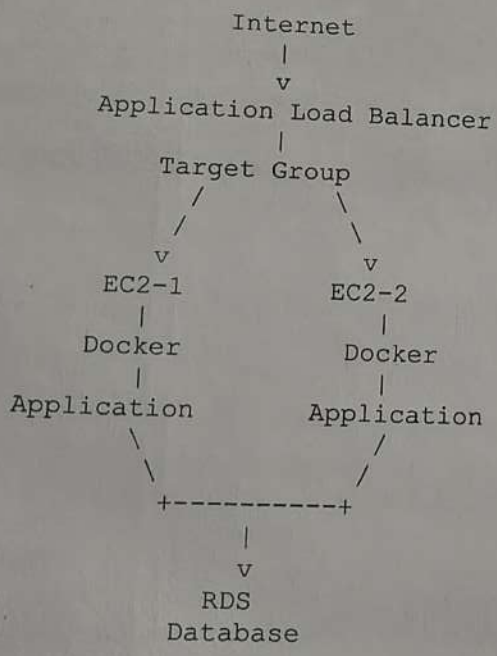
AWS DevOps Practical Assessment

Maximum Duration: 2 Hours
Environment: Fresh AWS Account
Assessment Type: Hands-on Practical

Candidates can pull sample code from the internet or GitHub. The objective is to deploy a production-style, Docker-based 3-tier application on AWS.

Common AWS Architecture

Your solution should follow this general architecture:



The application tier must be **stateless** so additional EC2 instances can launch automatically.

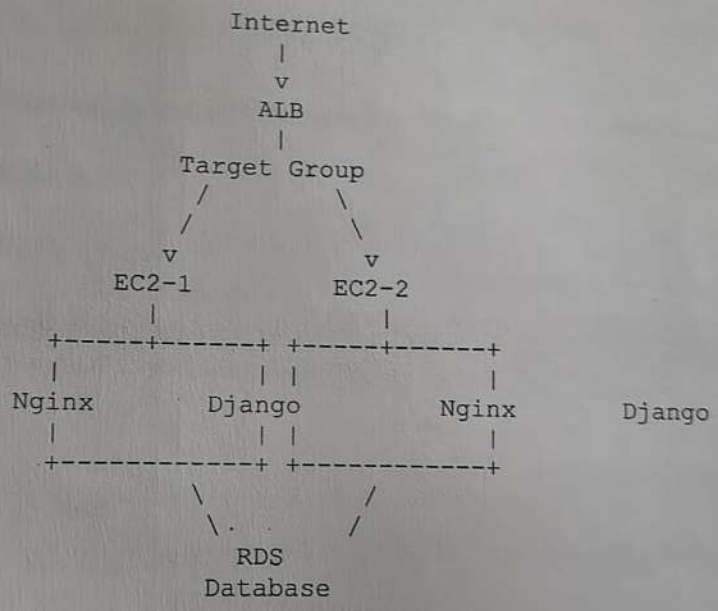
QUESTION 1 — Django 3-Tier Application

Objective

Deploy a Docker-based Django application on AWS with:

- 2 EC2 instances
- Docker
- Nginx reverse proxy
- Django application
- Database
- Application Load Balancer
- Auto Scaling Group
- CloudWatch
- Bash load-generation script

Required Architecture



Important

Nginx must work as a **reverse proxy** in front of the Django application.

The ALB should send HTTP traffic to the EC2 instance, and Nginx should forward the request to the Django application.

Task 1 — AWS Infrastructure

Create:

- Separate VPC/networking as required

- 2 EC2 instances
- Appropriate Security Groups
- Application Load Balancer
- Target Group
- Auto Scaling Group
- CloudWatch monitoring
- RDS database

The initial Auto Scaling configuration should be:

```
Minimum: 2
Desired: 2
Maximum: 4
```

Task 2 — Django Application

Deploy the provided Django application.

The application must:

- Run inside Docker.
- Connect to the database.
- Have a working health endpoint/page.
- Return a visible application response.

For example:

```
Hello from Django
```

Task 3 — Nginx Reverse Proxy

Configure Nginx so that:

```
ALB
 |
 v
Nginx
 |
 v
Django Application
```

The Django application should not need to be directly exposed to the internet.

Configure Nginx to forward requests to the Django application container.

Task 4 — Database

Use a separate database tier.

The database must not depend on the local EC2 filesystem.

Configure Django to connect to the database using configuration/environment variables.

Explain how the application instances can share the same database when ASG launches a new EC2 instance.

Task 5 — Load Balancer & Health Check

Configure the ALB and Target Group.

The Target Group health check must verify that the application is actually working.

Demonstrate:

- EC2-1 healthy
- EC2-2 healthy
- ALB DNS opens the application
- Traffic reaches the application through Nginx

Task 6 — Auto Scaling

Configure Auto Scaling based on an appropriate CloudWatch metric.

Initial state:

```
EC2-1
EC2-2
```

When the defined threshold is reached, the ASG should automatically launch another EC2 instance.

The new instance must automatically:

- Start

- Install/configure required components
- Start Docker/application
- Start Nginx
- Become healthy in the Target Group

You must not manually configure the third instance.

Task 7 — Bash Load Test

Create:

```
load-test.sh
```

The script should generate sufficient load to trigger your Auto Scaling policy.

Demonstrate:

```
2 EC2 Instances
       |
       v
Start load-test.sh
       |
       v
CPU/Application Load increases
       |
       v
CloudWatch
       |
       v
Auto Scaling
       |
       v
3rd EC2 Instance
       |
       v
Target Group
       |
       v
Healthy
       |
       v
ALB distributes traffic
```

We should see the new EC2 instance launch automatically.

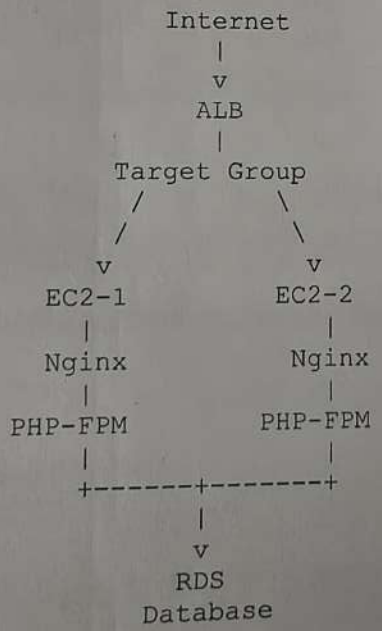
QUESTION 2 — PHP 3-Tier Application

Objective

Deploy a Docker-based PHP application on AWS using:

- 2 EC2 instances
- Docker
- Nginx
- PHP-FPM
- Database
- Application Load Balancer
- Auto Scaling Group
- CloudWatch
- Bash load-generation script

Required Architecture



Task 1 — AWS Infrastructure

Create:

- Required VPC/networking

- 2 EC2 instances
- Security Groups
- Application Load Balancer
- Target Group
- Auto Scaling Group
- CloudWatch monitoring
- RDS database

Auto Scaling:

```
Minimum: 2
Desired: 2
Maximum: 4
```

Task 2 — PHP Application

Deploy the provided PHP application.

The application must:

- Run using PHP-FPM.
- Run inside Docker.
- Connect to the database.
- Display a working application response.

For example:

```
Hello from PHP
```

The application should perform at least one database operation.

For example:

```
Database connection: OK
```

Task 3 — Nginx + PHP-FPM

Configure Nginx as the web server/reverse proxy.

The request flow should be:

```
ALB
 |
 v
Nginx
```

```
  |
  v
PHP-FPM
  |
  v
PHP Application
  |
  v
RDS
```

Nginx should pass PHP requests to PHP-FPM using the appropriate mechanism.

The PHP-FPM service should not be directly exposed to the internet.

Task 4 — Docker

Containerize the PHP application.

Your Docker configuration should:

- Build successfully.
- Start PHP-FPM.
- Work with Nginx.
- Communicate with the database.
- Allow the application to be deployed identically on both EC2 instances.

Explain your container architecture.

Task 5 — ALB & Target Group

Configure:

- Application Load Balancer
- Listener
- Target Group
- Health Check
- EC2 targets

Both EC2 instances must be healthy.

The application should be accessible through:

```
ALB DNS Name
```

The user should not directly access the EC2 application port.

Task 6 — Auto Scaling

Configure:

```
Minimum: 2
Desired: 2
Maximum: 4
```

Configure scaling based on an appropriate CloudWatch metric.

When load increases:

```
EC2-1 + EC2-2
       |
       v
High Load
       |
       v
CloudWatch
       |
       v
ASG
       |
       v
EC2-3
       |
       v
Application automatically configured
       |
       v
Target Group
       |
       v
ALB
```

The third EC2 instance must be created by the Auto Scaling Group.

Task 7 — Bash Load Test

Create:

```
load-test.sh
```

The script should generate sufficient load to trigger Auto Scaling.

1. Initial 2 EC2 instances.
2. Increased load.
3. CloudWatch metric increase.
4. ASG scale-out.
5. Third EC2 instance launch.
6. Application deployment on the new instance.
7. Target becoming healthy.
8. ALB distributing traffic.

Final Demonstration

For the assigned application, demonstrate the complete flow:

